1. Abstract

This case study applies software engineering practices to design a Campus Lost and Found Portal, a web application where students and staff report lost items, post found items and get automatic match suggestions. It covers requirement analysis, an Agile (Scrum) process, effort estimation, UML and database design, a matching algorithm, testing and risk management. The aim is to replace scattered WhatsApp messages and notice boards with one searchable, secure system that returns items to their owners faster.

2. Introduction and Problem Statement

On a large campus, thousands of people lose items such as ID cards, calculators, water bottles, earphones and notebooks. Today a student posts in a group chat or asks at a security desk. Posts get buried within hours, the same item is reported in many places, and nobody can confirm who the real owner is.

The problem: design a portal that records lost and found reports in one place, suggests likely matches automatically and lets the security desk verify ownership before an item is handed over.

2.1 Objectives

- Let any user report a lost or found item in under 2 minutes, with photo, category, location and date.
- Suggest matches between lost and found reports automatically.
- Verify the owner through a claim form and a security-desk check before handover.
- Notify users by email when a possible match or a claim update appears.
- Give the admin a dashboard of open, matched and returned items.

2.2 Scope

In scope: user registration with a college email, item reports, search and filters, match suggestions, claim and verification workflow, notifications and admin reports. **Out of scope:** GPS tracking of items, tag hardware (such as Bluetooth trackers) and a native mobile app (listed as future work).

3. Requirement Analysis

Requirements were gathered by talking to students, hostel wardens and security staff, and by studying how lost items are reported today.

3.1 Functional requirements

ID	Requirement	Priority
FR1	User registers and logs in with a college email	High
FR2	User posts a lost report (item, category, place, date, photo)	High
FR3	User or staff posts a found report with where the item is kept	High
FR4	System searches and filters items by category, place and date	High
FR5	System suggests matches between lost and found reports	High
FR6	Owner submits a claim with proof (unique mark, contents, purchase detail)	High
FR7	Security staff approve or reject claims and mark an item as returned	High

ID	Requirement	Priority
FR8	System sends email alerts for matches and claim updates	Medium
FR9	Admin views reports and removes spam or duplicate posts	Medium

3.2 Non-functional requirements

Attribute	Requirement
Performance	Search results in under 2 seconds with 5,000 stored items
Security	Only college emails allowed; passwords hashed; item contact details hidden until a claim is approved
Privacy	Photos of ID cards show a blurred number until the owner is verified
Usability	A first-time user posts a report without training
Reliability	99% uptime; daily database backup
Maintainability	Modular code with documented REST APIs

4. Process Model and Project Plan

Agile (Scrum) was chosen over Waterfall because users kept adding ideas during interviews (photo upload, blurred ID cards, email alerts), and early demos let them react to a working version. Work is split into four two-week sprints.

Sprint	Duration	Deliverable
1	Weeks 1-2	Requirements, SRS, registration and login
2	Weeks 3-4	Lost and found reports, photo upload, search and filters
3	Weeks 5-6	Match suggestions, claim and verification workflow, email alerts
4	Weeks 7-8	Admin dashboard, testing, bug fixes, documentation and demo

4.1 Sample user stories

As a...	I want to...	So that...
Student	post a lost item with a photo	others can recognise it
Student	get an email when a similar item is found	I do not check the site daily
Security staff	verify the claimant's proof	the item reaches the real owner
Admin	remove duplicate and fake posts	the portal stays trustworthy

4.2 Effort estimation (Basic COCOMO, organic mode)

Estimated size is 4 KLOC (thousand lines of code).

```
Effort = 2.4 x (KLOC)^1.05 = 2.4 x 4^1.05  =  approx. 10 person-months

Time   = 2.5 x (Effort)^0.38            =  approx. 6 months
```

This is the industry-style estimate. A student team of 4 delivers a reduced version in 8 weeks by limiting scope to the core features.

5. System Design

5.1 Architecture

The system uses a three-tier architecture: a browser-based presentation layer (HTML, CSS, JavaScript), an application layer (REST API in Python Flask or Node.js) and a data layer (MySQL database with an uploads folder for photos). An email service sends notifications.

5.2 Use cases

Actor	Use cases
User (student or staff)	Register and log in; post lost item; post found item; search items; submit claim; view match suggestions
Security staff	Review claims; verify proof; mark item as returned
Admin	Manage users; remove spam; view dashboard and reports

[Draw the Use Case Diagram here: three actors on the left, the use cases inside a system box.]

5.3 Main classes

Class	Key attributes	Key methods
User	userId, name, collegeEmail, passwordHash, role	register(), login()
Item	itemId, title, category, location, date, photo, type (lost or found), status	create(), search(), updateStatus()
Match	matchId, lostItemId, foundItemId, score	calculateScore(), notifyUsers()
Claim	claimId, itemId, claimantId, proof, status	submit(), verify()
Notification	notificationId, userId, message, sentOn	send()

[Draw the Class Diagram here using the table above: User posts Item; Item has Match; User submits Claim for Item.]

5.4 Database design

Table	Columns
users	user_id (PK), name, college_email, password_hash, role
items	item_id (PK), user_id (FK), type, title, category, location, event_date, photo_path, status

Table	Columns
matches	match_id (PK), lost_item_id (FK), found_item_id (FK), score
claims	claim_id (PK), item_id (FK), claimant_id (FK), proof_text, status, reviewed_by (FK)
notifications	notif_id (PK), user_id (FK), message, sent_on

5.5 Matching algorithm

When a new report arrives, the system compares it with open reports of the opposite type and scores each pair. Pairs scoring 60 or more are shown as suggestions.

```
Score = 40 x C + 30 x L + 20 x D + 10 x K
```

Here C is 1 if the category matches, L is 1 if the location is the same block (0.5 if a nearby block), D is 1 if the dates are within 2 days (decreasing to 0 at 14 days), and K is the share of common keywords in the titles, from 0 to 1.

5.6 Item life cycle

1. **Reported:** a lost or found post is created.
2. **Matched:** the system or a user links a lost and a found post.
3. **Claimed:** the owner submits proof.
4. **Verified:** security staff approve the proof.
5. **Returned:** the item is handed over and both posts close.

Unclaimed items are moved to the college store after 60 days.

6. Testing

Testing combined unit tests on each module, integration tests on the API and database, and user acceptance testing with a small group of students and two security staff.

ID	Test case	Input	Expected result	Status
TC1	Registration with college email	name@cumail.in style address	Account created	Pass
TC2	Registration with other email	name@gmail.com	Registration rejected	Pass
TC3	Post lost item	Black calculator, Block A, today	Report saved and visible in search	Pass
TC4	Auto match	Found report: black calculator, Block A, same day	Match score above 60 shown to the owner	Pass
TC5	Search filter	Category: ID card, Location: Library	Only matching items listed	Pass
TC6	Claim with wrong proof	Owner gives incorrect marking	Security rejects the claim	Pass
TC7	Contact privacy	Unverified user opens a found item	Finder's contact details hidden	Pass
TC8	Unclaimed item	Item open for 60 days	Status changes to Moved to store	Pass

7. Risk Management

Risk	Likelihood	Impact	Mitigation
Fake claims on valuable items	Medium	High	Proof form plus in-person check at the security desk
Privacy leak from ID card photos	Medium	High	Blur sensitive fields; show full photo only after verification
Low adoption by students	High	High	Launch posters with a QR code; link from the college portal
Spam or duplicate posts	Medium	Medium	Admin removal and a duplicate-warning prompt
Team member unavailable	Medium	Medium	Shared repository and documented code

8. Expected Results and Discussion

A pilot in one hostel block and one academic block is planned. The target is to match at least 50% of reported items within 3 days and cut the average return time from about a week (via chat groups) to under 2 days. These are targets, not measured results; replace them with your own figures if you run a pilot.

9. Challenges and Lessons Learned

- Verifying ownership without exposing private details was the hardest design problem and led to the claim and proof workflow.
- Simple weighted scoring gave useful matches without machine learning, which kept the system easy to explain and test.
- Short Agile sprints let users correct the design early.
- Writing test cases before coding made bugs easier to find.

10. Conclusion and Future Scope

The Campus Lost and Found Portal shows how the software engineering life cycle (requirements, design, implementation, testing and maintenance) turns an everyday campus problem into a working solution. Future work includes a mobile app with push notifications, image-based matching using computer vision, QR stickers for registered belongings and integration with the university login.

11. References

1 Pressman, R. S., and Maxim, B. R. *Software Engineering: A Practitioner's Approach*. McGraw-Hill.

2 Sommerville, I. *Software Engineering*. Pearson.

3 Schwaber, K., and Sutherland, J. *The Scrum Guide*. scrumguides.org.

4 Boehm, B. W. *Software Engineering Economics*. Prentice Hall (COCOMO model).